Uncovering Musical Patterns in Spotify Songs

CS 210 Final Project Report

Danial Javaid & Jeffrey Glasser

Demo Video | GitHub | Google Colab

1. Project Definition

Problem Statement

With how many songs are on music platforms such as Apple Music, YouTube, Soundcloud, and specifically the one used in this project, which is Spotify. It has been very complex to determine meaningful patterns between every individual song. There are a lot of factors contributing to the way a song sounds, such as timbre, loudness, tempo, acousticness, and energy. All of these attributes of a song don't always make it clear how these songs are grouped. So this project is aiming to eliminate that challenge by applying clustering techniques to Spotify song data in order to group music based on its musical characteristics. By applying these techniques, we'll see natural and clear patterns to define different types of songs based on the cluster they are put in based on the data. This can improve our understanding of musical structure and how tracks are similar to each other.

2. Novelty & Importance

Novelty

It introduces a more favorable approach instead of a more traditional one to understand music. People would normally assume certain songs are the same based on the genre attached to them, but understanding music is more complicated than that. Since genres are often more subjective and have a cultural bias, this project is far from something traditional. It's a more analytical approach by measuring musical attributes, which uncovers hidden structures that may not be visible to a normal person to point out within the music.

Importance

Understanding musical patterns as a whole is definitely important in these cases because the growth of Spotify has only been trending upwards. Improving music discoveries for people is definitely something that should be organized with the goal of trying to group songs together with their distinct interests. This project is also providing a bridge for people who coincide with music theory and data science by translating music qualities into new understandings, opening the door to deeper analytics on how songs are related to each other. So this project overall is providing an adaptable method for uncovering patterns in music.

3. Progress and Contribution

Data Utilization

As we've previously mentioned above, we are utilizing Spotify song data through the Spotify web API and the Spotify tracks dataset from Kaggle. This includes a list of audio features that we've already mentioned previously, which provides a strong foundation for using clustering techniques on this. This data is entered into the program using the Pandas library to make a database format, serving as our primary data structure for this project.

(Loading Data)

```
import pandas as pd
import sqlite3
import requests
import time

from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import f1_score, precision_score, recall_score, confusion_matrix, classification_report

import matplotlib.pyplot as plt

df = pd.read_csv('/content/spotify-tracks-dataset.csv')
df = df.drop(columns=["Unnamed: 0", "Unnamed: 0.1"])
```

(Cleaning The Data)

Data Cleaning

```
df2 = df.copy()

print("precleaning dimensions: ", df2.shape)

df2 = df2.drop_duplicates(subset=["track_id"])
print("after dropping duplicates: ", df2.shape)

audio_features = [
    "danceability",
    "energy",
    "loudness",
    "speechiness",
    "acousticness",
    "instrumentalness",
    "valence",
    "tempo"
]

df2 = df2.dropna(subset=audio_features)
print("after dropping rows with missing audio features: ", df2.shape)

omitted_features = [
    "artists",
    "album_name",
    "track_name",
    "track_genre",
    "track_id"
]

df2 = df2.drop(columns=omitted_features)
print("after dropping omitted features: ", df2.shape)
```

```
precleaning dimensions:  (64071, 22)
after dropping duplicates:  (54200, 22)
after dropping rows with missing audio features:  (54200, 22)
after dropping omitted features:  (54200, 17)
```

Cleaning The Data: The above shows what we had to do before applying the clustering techniques. First, we removed duplicate songs from the dataset, then we removed rows we didn't care about, so incomplete data wouldn't distort our results. Lastly, we dropped columns we didn't need, since we only needed the numerical features of each song. Handling redundancy, missing values, and removing the non-essential attributes of each song. We

ensured that the data is well structured enough to apply clustering techniques to get an analysis.

Data Storage

A major challenge we faced was storing the data from the API calls. Since we needed data for over 80k tracks, retrieving it from the Spotify Web API took about 12 minutes each time. The time efficient solution to this problem was to call the API once and store the results in a SQLite database. That database is then merged to the dataframe we downloaded from Kaggle. It is that database that is referred to every session when we have to run the cells again. This saved us a lot of time and prevented any further problems with the API. There was also the issue of the data not persisting after a session. To fix this, we had the Colab connect to the user's Google Drive, and search for the database and data set from their drive. This was one of the limitations of Google Colab we found and will keep in mind for the future.

Models, Techniques, & Algorithms

This dataset overall was first structured into input features and target labels, where the audio features above, such as danceability, energy, loudness, speechiness, acousticness, instrumentalness, valence, and tempo. These are all serving as predictors for the overall results, while the target variable is the song's respective decade.

```
X = current_df[audio_features]
y = current_df["decade"]
```

Then we split the dataset into training and testing sets using an 80/20 split to ensure proper model evaluation. This allows the models to be trained on one portion of the data while being evaluated on hidden data, providing a more accurate measure of performance.

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
print("training set size:", len(X_train))
print("test set size:", len(X_test))
print(10840/54200)
```

```
training set size: 4676
test set size: 1169
0.2
```

The model is then trained on the scaled training data and used to generate predictions on the test set. Then several metrics were calculated, including F1-score, precision, and recall. Providing a more comprehensive understanding of performance.

Training Logistic Model

```
lr_model = LogisticRegression(
    max_iter=1000,
    random_state=42,
    class_weight="balanced" # because 2010 dominates the sample
)
# fitting and predicting
lr_model.fit(X_train_scaled, y_train)
lr_predictions = lr_model.predict(X_test_scaled)

# accuracy testing
lr_f1 = f1_score(y_test, lr_predictions, average="macro")
lr_precision = precision_score(y_test, lr_predictions, average="macro")
lr_recall = recall_score(y_test, lr_predictions, average="macro")

print("f1_score: ", lr_f1)
print("lr_precision: ", lr_precision)
print("lr_recall: ", lr_recall)
print(classification_report(y_test, lr_predictions))
```

```
f1_score:  0.30843359864743214
lr_precision:  0.30790398232671545
lr_recall:  0.3294289591244136
              precision    recall  f1-score   support

        1970       0.35      0.51      0.42       224
        1980       0.31      0.36      0.33       222
        1990       0.20      0.08      0.12       231
        2000       0.29      0.22      0.25       242
        2010       0.38      0.47      0.42       250

    accuracy                           0.33      1169
   macro avg       0.31      0.33      0.31      1169
weighted avg       0.31      0.33      0.31      1169
```

Experimental Design

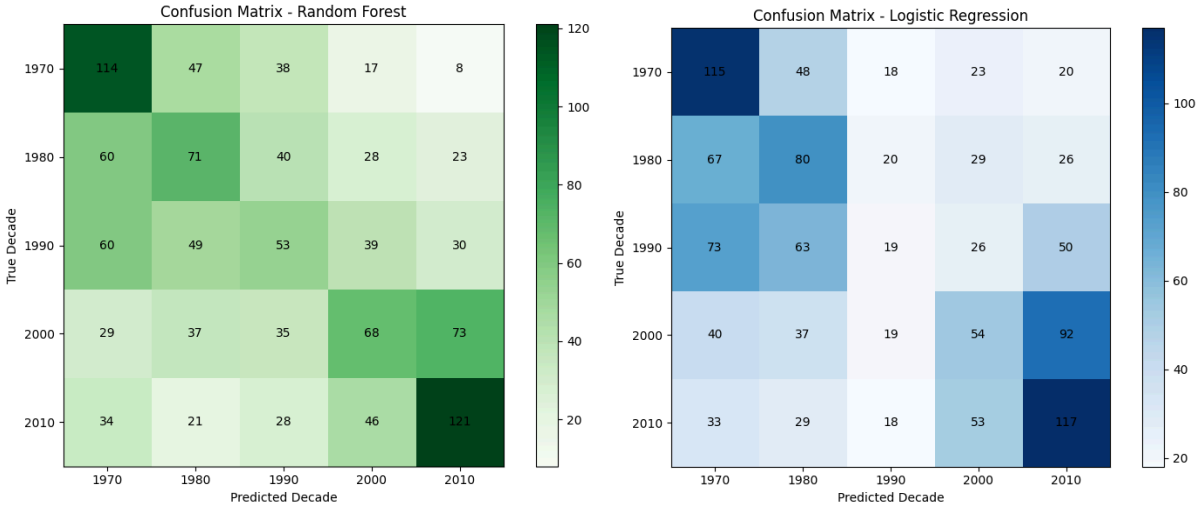
We predicted that specific combinations of audio features of a song would have stronger relationships with the decade it would belong to. In particular, we were expecting features such as energy, loudness, and tempo to carry most of the identity of a song's decade since music production styles have changed over time. While other features will have at least some influence on the outcome, we've come to believe that the majority of predictive power could come from features that reflect the overall intensity and production quality of a track.

Key Finding, Results, & Evaluation

The table below shows the results of accuracy tests for models with different tweaks. We tested how the accuracy changes with class_weight="balanced" and using the balanced dataset. The model was trained on scaled features and evaluated using macro-average metrics to ensure equal weighting across all decade classes.

Model	Macro F1	Precision	Recall	Accuracy
LR (unbalanced, no undersample)	0.160	0.265	0.204	0.59
RF (unbalanced, no undersample)	0.254	0.401	0.254	0.60
LR (balanced, no undersample)	0.248	0.268	0.341	0.39
RF (balanced, no undersample)	0.359	0.358	0.365	0.37
LR (unbalanced, undersample)	0.309	0.309	0.329	0.33
RF (unbalanced, undersample)	0.347	0.347	0.354	0.36
LR (balanced, undersample)	0.308	0.308	0.329	0.33
RF (balanced, undersample)	0.359	0.358	0.365	0.37

This model achieved a macro F1-score of 0.36, with a macro precision of 0.36 and a macro recall of 0.36, which indicates that it was a moderate performance overall. The overall accuracy was 0.37, meaning the model predicted the decade of a song correctly a little over one-third of the time. The performance was varying across the decades, with the 2010s achieving the strongest results with an F1-score of 0.48, followed by the 1970s with an F1-score of 0.44. Indicating that these decades have more unique and distinguishable audio features than the rest of the decades. As for those other decades, though, they performed more poorly with F1-Scores ranging from 0.25 to 0.32, which suggests that there is overlap in music quality between these eras of music. Evaluation-wise, these results suggest that the model only captures a moderate temporal signal from Spotify audio features. There are definitely differences between how the decades are spread in terms of their audio features, but these differences aren't strong enough to determine very accurate predictions. With many songs from neighboring decades sounding very similar to each other, it makes classification that much more difficult.



Now for the confusion matrices, shown above are two different matrices generated for both Logistic Regression and Random Forest models. Helping us visualize how often songs from each decade were correctly classified or unclassified into other decades. In the Logistic Regression matrix, there is tons of overlap with the 1990s and 2000s with them not having a majority in the diagonal in the right predicated decade, indicating this model has difficulties capturing the complexities of audio features. Let alone this model was more accurate in the 1970s and 2010s, but the 1980s was more of like a spread despite the majority of the songs being predicted correctly overall. For the Random Forest model, it shows a little bit of improvement in classifying particularly for the 1970s and the 2010s, where more predictions are actually falling on the diagonal in this matrix. But clearly as we see, it isn't as perfect as it may be described, only ever really having a couple of minor improvements which is very much similar to the Logistic Regression model. It is still showing confusion between the adjacent decades especially towards the middle of this matrix. Suggesting that even though the Random Forest matrix can capture more complex relationships, the overlap between all of the decades still remains a challenge. These matrices are both trying to highlight the same issue, audio features will obviously not be enough to distinguish a song's decade overall. With the Random

Forest improving the specifics of the issue slightly better, both of the models show consistent misclassification between adjacent periods, reinforcing the idea that the evolution of music is gradual rather than strictly defined.

Advantages & Limitations

An advantage to this task is that it is less likely to overfit compared to a single decision tree. Using class_weight="balanced" helps reduce the overall bias toward any one decade, making the results far more fair. But there's definitely also some clear limitations, with the model's performance only really being moderate; it shows that it is struggling to separate decades since they share so many similarities in audio features. And over time, it'll be even harder to distinguish the differences in music to establish which decade they fit based on their audio features. Another limitation to these results is that audio features are not technically the reason you define a song's decade. There are other factors surrounding that topic, whether it'd be cultural trends or production style; those factors aren't included in this analysis, only seeing partial patterns based on what we know from the data.

4. Changes After Proposal

Differences From Proposal

This project remained consistent with what was said in the proposal to be implemented throughout this project. Both training models were used, being the Multinomial Logistic Regression and Random Forest, the Logistic Regression model was used as a baseline for this part, and its results helped show how a simpler linear model performed. The Random Forest model was then used to compare all of the features to show all of the complex relationships in a simplistic model. A key change that we did in this project was our database choice; we originally were going to use PostgreSQL. We then proceeded to pivot over to SQLite; the familiarity was more there for us to understand. Plus, we had a small number of tables with simple queries to go along with it, and a heavy-duty database like PostgreSQL was not needed. The workflow with this was way simpler since nothing we had to do was necessarily complex overall throughout the process. Data collection was heavily reliant on the Spotify Web API to get all of the release dates, but in practice, this was more or less simplified since the dataset had already contained information after preprocessing. So overall, we followed the plan closely, with the only real difference being the simplification of the database, the data collection,

5. Conclusion

This project was done to determine whether Spotify's audio features can accurately put songs into their respective decade. Using both of the models above, we see how well these audio features can show differences in musicality in eras. Even if the models were able to produce results at the very least, they were considered moderate with the F1- score being a 0.36, and with certain decades being more distinguishable than others. Some decades were overlapping with each other which made it more confusing to determine where the boundaries were, but this also showed that music was evolving over time. Highlighting the strengths and limitations of

machine learning for this type of project, with the models capturing the general trends they were limited by what they had to go off of and did not fully account for any influences that could possibly define different eras of music. But overall this project signifies that Spotify's audio features can point us in a direction for identifying music decades. But with not the highest accurate conclusions, future improvements can be made to capture better complexity on how music is changed over time.